

OASYS Backend Component

---

# OasysSettings

---

*Author:* Eric J. FRANCOIS

May 29, 2026

# 1 description

The `OasysSettings` class is a Singleton responsible for loading, managing, and distributing system-wide configuration, languages, skins, and module properties. It aggregates default configurations, merges them with overrides persisted in the database, and safely exposes them to the backend and frontend.

Additionally, this class manages dynamic user-specific configurations, applying user group capabilities (such as editor buttons) and validating interface languages upon login.

## 2 usage

Initialization typically happens at the bootstrap level via the main `OasysApp` class instance, which is required by the constructor. After that, it can be retrieved from the public property `config` of the `OasysApp` instance.

### Example

Fetching a specific property:

```
$maxUpload = $settings->getProperty('upload_max_filesize');
```

Loading user-specific settings after login:

```
$settings->fetchUserSettings($userId, $forceLang, $userGroup);
```

### Dependencies:

This class relies on the `OasysApp` class for initial injection, `rixPDO` for database interactions, and `settingsDefault.php` for its baseline configurations.

## 3 public properties

The `OasysSettings` instance maintains several public arrays that contain the runtime state of the application's configuration.

|                          |                                                                                                                                    |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>\$settings</code>  | The associative array containing all currently active merged settings (defaults combined with database values and user overrides). |
| <code>\$languages</code> | Available system languages fetched from the <code>languages</code> table.                                                          |
| <code>\$skins</code>     | Available skins parsed dynamically from the file system (base and module skins).                                                   |

## 4 instance methods

The following public methods are available on an `OasysSettings` instance.

|                                      |                                                                                             |
|--------------------------------------|---------------------------------------------------------------------------------------------|
| <code>getProperty(\$property)</code> | Returns the value of a specific setting by its key.                                         |
| <code>getSettingsArray()</code>      | Returns a reference to the main <code>\$settings</code> array.                              |
| <code>getDefaults()</code>           | Returns the base default settings array before any database or user overrides were applied. |

|                                     |                                                                                                                                           |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>getJSSafeSettings()</code>    | Returns an array of settings specifically filtered (excluding ones marked <code>noJS</code> ) to be safely passed to the frontend client. |
| <code>getLanguages()</code>         | Returns the array of active languages.                                                                                                    |
| <code>getSkins()</code>             | Returns the array of available skins.                                                                                                     |
| <code>fetchUserSettings(...)</code> | Fetches and merges user-specific database overrides and aggregates editor button permissions across a user's assigned groups.             |
| <code>getIntLang(...)</code>        | Determines the correct interface language based on user preference, forced input, or browser HTTP headers.                                |
| <code>getDatabaseInstance()</code>  | Returns the <code>rixPDO</code> instance injected during construction.                                                                    |

## 4.1 fetchUserSettings parameters

|                           |               |                                                                                   |
|---------------------------|---------------|-----------------------------------------------------------------------------------|
| <code>\$userId*</code>    | <i>string</i> | The unique identifier of the user.                                                |
| <code>\$userGroup*</code> | <i>array</i>  | An array of user group IDs utilized to parse aggregate access permissions.        |
| <code>\$forceLang</code>  | <i>string</i> | An optional language code to force (e.g., from the login screen or query string). |

\*mandatory parameters

## 5 static methods

|                                        |                                                                                                                                 |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>getInstance(\$app)</code>        | Returns the singleton instance. If it doesn't exist, it requires an <code>OasysApp</code> instance to initialize.               |
| <code>findSkins()</code>               | Scans the file system (both base skins and module skins) to parse <code>properties.json</code> files and build the skins array. |
| <code>findMods()</code>                | Scans the <code>modules</code> folder to find and parse installed module capabilities via their <code>editor.json</code> files. |
| <code>establishRootURL()</code>        | Determines the absolute web root URL based on server configuration.                                                             |
| <code>establishJSRootURL(\$url)</code> | Modifies the root URL to account for server context prefixes, primarily used for linking JavaScript and CSS assets.             |